

**PRAKTIKUM SISTEM CERDAS DAN PENDUKUNG KEPUTUSAN
SEMESTER GENAP TA 2025/2026**

LAPORAN PROYEK AKHIR



DISUSUN OLEH:

NIM	:	123240024 123240200
NAMA	:	Nayla Cahaya Zahira Rifki Zakaria Yahya
PLUG	:	IF-I
NAMA ASISTEN	:	Hawla Khufyatul Qolbu Khatama Putra

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

LAPORAN PROYEK AKHIR

Disusun oleh:

Nayla Cahaya Zahira

123240024

Rifki Zakaria Yahya

123240200

Telah Diperiksa dan Disetujui oleh Asisten Praktikum Sistem Cerdas dan Pendukung
Keputusan

Pada Tanggal:

Asisten Praktikum

Asisten Praktikum

Hawla Khufyatul Qolbu
NIM. 123230021

Khatama Putra
NIM. 123230053

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa yang senantiasa mencurahkan rahmat dan hidayah-Nya sehingga saya dapat menyelesaikan praktikum Sistem Cerdas dan Pendukung Keputusan serta laporan proyek akhir praktikum yang berjudul Sistem Pendukung Keputusan Berbasis AHP untuk Optimasi Keputusan Restocking Produk bagi Manajer Retail Indonesia. Adapun laporan ini berisi tentang proyek akhir yang saya pilih dari hasil pembelajaran selama praktikum berlangsung.

Tidak lupa ucapan terimakasih kepada asisten praktikum yang selalu membimbing dan mengajari saya dalam melaksanakan praktikum dan dalam menyusun laporan ini. Laporan ini masih sangat jauh dari kesempurnaan, oleh karena itu kritik serta saran yang membangun saya harapkan untuk menyempurnakan laporan akhir ini.

Atas perhatian dari semua pihak yang membantu penulisan ini, saya ucapkan terimakasih. Semoga laporan ini dapat dipergunakan seperlunya.

Yogyakarta, 25 Mei 2026

Penyusun 1,

Penyusun 2,

Nayla Cahaya Zahira

NIM. 123240024

Rifki Zakaria Yahya

NIM. 123240200

DAFTAR ISI

HALAMAN PENGESAHAN	ii
KATA PENGANTAR.....	iii
DAFTAR ISI	iv
BAB I PENDAHULUAN	1
1.1 Latar Belakang Masalah	1
1.2 Tujuan Proyek Akhir	1
1.3 Manfaat Proyek Akhir	1
BAB II PEMBAHASAN.....	3
2.1 Dasar Teori	3
2.2 Skenario Kasus & Dataset	3
2.3 Pemodelan Sistem Pendukung Keputusan	4
2.4 Perhitungan & Algoritma	5
2.5 Implementasi & Tampilan Sistem	5
BAB III JADWAL Pengerjaan DAN PEMBAGIAN TUGAS	18
3.1 Jadwal Pengerjaan	18
3.2 Pembagian Tugas.....	18
BAB IV KESIMPULAN DAN SARAN.....	19
4.2 Kesimpulan.....	19
4.3 Saran	19
DAFTAR PUSTAKA.....	20

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Industri retail di Indonesia mengalami pertumbuhan yang pesat seiring dengan meningkatnya konsumsi masyarakat dan berkembangnya berbagai saluran penjualan, baik secara offline maupun online. Di tengah persaingan yang semakin ketat, salah satu tantangan utama yang dihadapi oleh para manajer retail adalah pengambilan keputusan terkait manajemen stok produk. Keputusan tentang produk mana yang perlu di-restock terlebih dahulu seringkali didasarkan pada intuisi semata, tanpa mempertimbangkan berbagai faktor secara objektif dan terukur.

Kesalahan dalam menentukan prioritas restocking dapat berdampak langsung pada kerugian bisnis, seperti kehabisan stok produk unggulan yang berpotensi kehilangan pendapatan, atau penumpukan stok produk yang kurang diminati sehingga meningkatkan biaya operasional. Oleh karena itu, dibutuhkan sebuah sistem yang mampu membantu manajer dalam membuat keputusan restocking secara lebih sistematis, objektif, dan berbasis data.

Proyek akhir ini mengembangkan Sistem Pendukung Keputusan (SPK) menggunakan metode Analytical Hierarchy Process (AHP) untuk menentukan prioritas restocking produk pada bisnis retail di Indonesia. Metode AHP dipilih karena mampu menangani pengambilan keputusan multi-kriteria dengan mempertimbangkan bobot kepentingan tiap kriteria secara konsisten dan terukur. Dataset yang digunakan merupakan data transaksi retail Indonesia yang bersumber dari Kaggle, terdiri dari 55.000 transaksi dengan 26 jenis produk dalam 5 kategori. Sistem dibangun dalam bentuk antarmuka grafis menggunakan Streamlit agar dapat digunakan secara interaktif oleh pengguna.

1.2 Tujuan Proyek Akhir

Tujuan dari proyek akhir ini adalah sebagai berikut:

1. Membangun Sistem Pendukung Keputusan (SPK) berbasis metode AHP untuk membantu manajer retail dalam menentukan prioritas restocking produk secara objektif dan terukur.
2. Mengimplementasikan algoritma AHP untuk menghitung bobot kriteria dan skor akhir setiap alternatif produk berdasarkan data transaksi retail Indonesia.
3. Menyajikan hasil perbandingan produk dalam antarmuka grafis interaktif berbasis Streamlit yang dilengkapi dengan visualisasi data dan proses perhitungan AHP yang transparan.

1.3 Manfaat Proyek Akhir

Manfaat yang diharapkan dari proyek akhir ini adalah sebagai berikut:

1. Bagi Manajer Retail: Sistem ini membantu pengambilan keputusan restocking menjadi lebih cepat, konsisten, dan berbasis data, sehingga mengurangi risiko

kehabisan stok produk unggulan maupun penumpukan stok produk yang kurang laku.

2. Bagi Bisnis Retail: Optimasi restocking berdasarkan profitabilitas dan performa penjualan dapat meningkatkan efisiensi operasional dan memaksimalkan pendapatan toko.
3. Bagi Akademis: Proyek ini memberikan contoh penerapan nyata metode AHP dalam domain retail Indonesia, yang dapat menjadi referensi bagi penelitian atau pengembangan SPK selanjutnya.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Analytical Hierarchy Process (AHP) adalah metode pengambilan keputusan multi-kriteria yang dikembangkan oleh Thomas L. Saaty pada tahun 1980. AHP bekerja dengan cara mendekomposisi permasalahan yang kompleks menjadi sebuah hierarki yang terdiri dari tujuan (goal), kriteria, dan alternatif. Keunggulan AHP dibandingkan metode lain adalah kemampuannya mengukur konsistensi penilaian pengambil keputusan melalui Consistency Ratio (CR).

Langkah-langkah utama dalam metode AHP adalah sebagai berikut:

1. Mendefinisikan masalah dan menyusun hierarki keputusan yang terdiri dari tujuan, kriteria, dan alternatif.
2. Membuat matriks perbandingan berpasangan (pairwise comparison matrix) menggunakan skala Saaty 1-9, di mana nilai 1 berarti sama penting, 3 sedikit lebih penting, 5 lebih penting, 7 sangat lebih penting, dan 9 mutlak lebih penting. Nilai kebalikan (reciprocal) digunakan untuk perbandingan sebaliknya.
3. Menghitung vektor prioritas (bobot kriteria) dengan cara menormalisasi matriks perbandingan berpasangan, yaitu membagi setiap elemen dengan jumlah kolomnya, kemudian menghitung rata-rata tiap baris.
4. Menguji konsistensi penilaian menggunakan Consistency Ratio (CR) dengan rumus $CR = CI / RI$, di mana CI (Consistency Index) = $(\lambda_{max} - n) / (n - 1)$ dan RI (Random Index) merupakan nilai indeks acak berdasarkan jumlah kriteria. Penilaian dianggap konsisten jika $CR < 0,10$.
5. Menghitung skor akhir setiap alternatif dengan mengalikan nilai ternormalisasi tiap kriteria dengan bobot kriteria yang diperoleh, kemudian menjumlahkan hasilnya. Alternatif dengan skor tertinggi merupakan alternatif terbaik.

2.2 Skenario Kasus & Dataset

Skenario kasus dalam proyek ini adalah pengambilan keputusan restocking produk pada bisnis retail Indonesia. Pengambil keputusan adalah manajer toko retail yang harus menentukan produk mana yang perlu diprioritaskan untuk diisi ulang stoknya dari 26 jenis produk yang tersedia. Tujuan akhir dari sistem ini adalah menghasilkan peringkat produk dari yang paling diprioritaskan hingga yang paling rendah prioritasnya untuk di-restock, berdasarkan kombinasi beberapa kriteria kinerja penjualan.

Dataset yang digunakan dalam proyek ini adalah dataset Retail Indonesia 55K yang bersumber dari platform Kaggle (<https://www.kaggle.com>). Dataset ini berisi 55.000 baris data transaksi penjualan retail di Indonesia sepanjang tahun 2024, dengan total 37 kolom atribut. Dataset mencakup 26 jenis produk yang terbagi dalam 5 kategori, yaitu Aksesoris, Footwear, Apparel, Home, dan Stationery, yang dijual di berbagai kota di Indonesia melalui berbagai saluran penjualan.

Untuk keperluan SPK, data mentah diagregasi per produk sehingga menghasilkan 26 baris data alternatif. Setiap baris merepresentasikan satu jenis produk dengan nilai agregat dari seluruh transaksinya, meliputi total penjualan, rata-rata profit, rata-rata margin, total kuantitas terjual, dan rata-rata diskon.

2.3 Pemodelan Sistem Pendukung Keputusan

Pemodelan SPK dalam proyek ini menggunakan pendekatan AHP dengan struktur hierarki sebagai berikut: tujuan utama adalah menentukan prioritas restocking produk, dengan 5 kriteria penilaian dan 26 alternatif produk.

Daftar Kriteria:

C1 - Total Penjualan (Benefit): Total akumulasi omzet penjualan suatu produk. Semakin tinggi nilai ini, semakin tinggi permintaan pasar terhadap produk tersebut, sehingga prioritas restocking semakin tinggi.

C2 - Rata-rata Profit (Benefit): Rata-rata keuntungan bersih per transaksi dari suatu produk. Produk dengan profit rata-rata tinggi lebih menguntungkan secara finansial sehingga diprioritaskan untuk di-restock.

C3 - Rata-rata Margin Persen (Benefit): Persentase margin keuntungan rata-rata per transaksi. Margin yang tinggi menunjukkan efisiensi bisnis yang baik.

C4 - Total Kuantitas Terjual (Benefit): Jumlah total unit produk yang berhasil terjual. Produk dengan kuantitas terjual tinggi berarti cepat habis dan membutuhkan restocking yang lebih segera.

C5 - Rata-rata Diskon Persen (Cost): Rata-rata persentase diskon yang diberikan pada suatu produk. Diskon besar mengindikasikan produk tersebut kurang laku tanpa insentif harga, sehingga prioritas restockingnya lebih rendah.

Daftar Alternatif (26 Produk): Totebag Kanvas, Sandal Kulit, Kaos Polos Katun Lokal, Tumbler Stainless, Dress Batik Modern, Topi Snapback, Polo Shirt Premium, Sticky Notes Lucu, Bantal Sofa Batik, Ikat Pinggang Kulit, Sepatu Sneakers Lokal, Jaket Bomber, Tas Ransel Laptop, Celana Chino, Kaus Kaki Motif, Dompot Kulit Pria, Jam Tangan Casual, Kemeja Flanel, Notebook Hardcover, Sepatu Formal Pria, Tote Bag Eco, Rok Midi Batik, Hoodie Polos, Mug Keramik Custom, Tempat Pensil Unik, Kacamata Fashion.

Skala penilaian yang digunakan dalam perbandingan berpasangan AHP mengacu pada Skala Saaty 1-9: 1 (sama penting), 3 (sedikit lebih penting), 5 (lebih penting), 7 (sangat lebih penting), 9 (mutlak lebih penting), serta nilai reciprocal $1/3$, $1/5$, $1/7$, $1/9$ untuk perbandingan sebaliknya. Normalisasi nilai alternatif menggunakan metode Min-Max dengan penyesuaian tipe kriteria (benefit: nilai tinggi lebih baik; cost: nilai rendah lebih baik).

(Jika Fuzzy: Jabarkan variabel input dan output, batasan nilai (*range*) setiap himpunan fuzzy (misal: Rendah [0-5], Sedang [4-7], dsb.), dan daftar *Rule Base*.)

2.4 Perhitungan & Algoritma

Berikut adalah langkah-langkah perhitungan algoritma AHP yang diterapkan dalam sistem ini:

Langkah 1 - Pembentukan Matriks Perbandingan Berpasangan. Pengguna mengisi nilai perbandingan antar kriteria pada matriks berukuran $n \times n$ ($n = 5$ kriteria). Nilai diagonal bernilai 1 (setiap kriteria sama penting dengan dirinya sendiri), dan nilai di bawah diagonal merupakan kebalikan (reciprocal) dari nilai di atas diagonal. Contoh: jika C1 dibandingkan C2 = 3, maka C2 dibandingkan C1 = $1/3$.

Langkah 2 - Normalisasi Matriks. Setiap elemen matriks dibagi dengan jumlah total kolomnya. Hasil normalisasi menghasilkan matriks baru di mana jumlah tiap kolom bernilai 1.

Langkah 3 - Perhitungan Vektor Prioritas (Bobot). Rata-rata setiap baris dari matriks ternormalisasi dihitung sebagai bobot (priority vector) masing-masing kriteria. Jumlah seluruh bobot = 1.

Langkah 4 - Uji Konsistensi. Nilai $\lambda_{\max}$ dihitung dari perkalian matriks awal dengan vektor prioritas dibagi vektor prioritas. $CI = (\lambda_{\max} - n) / (n - 1)$. $CR = CI / RI$, di mana RI untuk $n=5$ adalah 1,12. Jika $CR < 0,10$ maka penilaian konsisten dan dapat dilanjutkan.

Langkah 5 - Normalisasi Nilai Alternatif. Nilai setiap kriteria untuk masing-masing alternatif dinormalisasi menggunakan metode Min-Max. Untuk kriteria benefit: $(\text{nilai} - \min) / (\max - \min)$. Untuk kriteria cost: $(\max - \text{nilai}) / (\max - \min)$. Hasil normalisasi berada pada rentang 0 sampai 1.

Langkah 6 - Perhitungan Skor Akhir. Skor akhir tiap alternatif dihitung dengan rumus: $\text{Skor} = \sum (\text{bobot kriteria } C_i \times \text{nilai ternormalisasi alternatif pada } C_i)$. Alternatif diurutkan berdasarkan skor dari tertinggi ke terendah sebagai rekomendasi prioritas restocking.

2.5 Implementasi & Tampilan Sistem

```
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

# _____
# CONFIG
# _____

st.set_page_config(
    page_title="SPK AHP - Optimasi Restocking Produk Retail",
    page_icon="",
    layout="wide",
    initial_sidebar_state="expanded",
)

# _____
# AHP HELPERS
# _____

SAATY_RI = {1: 0.00, 2: 0.00, 3: 0.58, 4: 0.90,
            5: 1.12, 6: 1.24, 7: 1.32, 8: 1.41, 9: 1.45}
```

```

SAATY_SCALE = {
    "1 - Sama penting": 1,
    "3 - Sedikit lebih penting": 3,
    "5 - Lebih penting": 5,
    "7 - Sangat lebih penting": 7,
    "9 - Mutlak lebih penting": 9,
    "1/3": 1/3,
    "1/5": 1/5,
    "1/7": 1/7,
    "1/9": 1/9,
}

def ahp_weights(matrix: np.ndarray):
    """Return (weights, lambda_max, CI, CR)."""
    n = matrix.shape[0]
    col_sum = matrix.sum(axis=0)
    norm = matrix / col_sum
    weights = norm.mean(axis=1)
    weighted_sum = matrix @ weights
    lambda_vals = weighted_sum / weights
    lambda_max = lambda_vals.mean()
    ci = (lambda_max - n) / (n - 1) if n > 1 else 0
    ri = SAATY_RI.get(n, 1.49)
    cr = ci / ri if ri != 0 else 0
    return weights, lambda_max, ci, cr

def normalize_minmax(df: pd.DataFrame, cols_benefit: list, cols_cost:
list) -> pd.DataFrame:
    out = df.copy()
    for c in cols_benefit:
        mn, mx = df[c].min(), df[c].max()
        out[c] = (df[c] - mn) / (mx - mn) if mx != mn else 0
    for c in cols_cost:
        mn, mx = df[c].min(), df[c].max()
        out[c] = (mx - df[c]) / (mx - mn) if mx != mn else 0
    return out

# -----
# LOAD DATA
# -----
@st.cache_data
def load_data():
    df = pd.read_csv("retail_indonesia_55k.csv")
    return df

@st.cache_data
def aggregate_by_product(df):
    grp = df.groupby("nama_produk").agg(
        total_penjualan = ("total_penjualan", "sum"),
        rata_profit      = ("profit", "mean"),
        rata_margin      = ("margin_persen", "mean"),
        total_qty        = ("qty", "sum"),
        rata_diskon      = ("diskon_persen", "mean"),
        kategori         = ("kategori", "first"),
    ).reset_index()
    grp.columns.name = None
    return grp

```

```

df_raw = load_data()
df_prod = aggregate_by_product(df_raw)

CRITERIA = {
    "C1 - Total Penjualan": "total_penjualan",
    "C2 - Rata-rata Profit": "rata_profit",
    "C3 - Rata-rata Margin (%)": "rata_margin",
    "C4 - Total Qty Terjual": "total_qty",
    "C5 - Rata-rata Diskon (%)": "rata_diskon",
}
CRITERIA_COLS = list(CRITERIA.values())
CRITERIA_LABELS = list(CRITERIA.keys())
N = len(CRITERIA)

# -----
# SIDEBAR NAVIGATION
# -----
st.sidebar.image(
    "Logo.png",
    width=250,
)
st.sidebar.title("SPK Restocking Retail")
st.sidebar.caption("Sistem Pendukung Keputusan Berbasis AHP\nOptimasi Keputusan Restocking Produk\nbagi Manajer Retail Indonesia")

page = st.sidebar.radio(
    "Navigasi",
    ["Dataset", "Hitung SPK", "Hasil & Grafik"],
    index=0,
)

st.sidebar.markdown("---")
st.sidebar.markdown("***Jumlah Data**")
st.sidebar.metric("Total Transaksi", f"{len(df_raw):,}")
st.sidebar.metric("Jumlah Produk", df_prod.shape[0])
st.sidebar.metric("Jumlah Kriteria", N)

# =====
# PAGE 1 - DATASET
# =====
if page == "Dataset":
    st.title("Dataset Transaksi Retail Indonesia")
    st.markdown(
        "Dataset berisi **55.000 transaksi** retail dari berbagai kota di Indonesia. "
        "Untuk keperluan SPK restocking, data diagregasi per produk menjadi **26 alternatif** "
        "yang akan dievaluasi kelayakannya untuk diprioritaskan dalam pengadaan stok."
    )

    tab1, tab2 = st.tabs(["Data Mentah (Raw)", "Data Agregat per Produk"])

    with tab1:
        st.subheader("Data Transaksi Mentah")

        col_filter1, col_filter2 = st.columns(2)

```

```

        with col_filter1:
            kat_opts = ["Semua"]
sorted(df_raw["kategori"].dropna().unique().tolist())
            kat_sel = st.selectbox("Filter Kategori", kat_opts)
        with col_filter2:
            kota_opts = ["Semua"]
sorted(df_raw["kota"].dropna().unique().tolist())
            kota_sel = st.selectbox("Filter Kota", kota_opts)

df_view = df_raw.copy()
if kat_sel != "Semua":
    df_view = df_view[df_view["kategori"] == kat_sel]
if kota_sel != "Semua":
    df_view = df_view[df_view["kota"] == kota_sel]

st.caption(f"Menampilkan {len(df_view):,} baris")
st.dataframe(df_view.head(500), use_container_width=True,
height=420)
st.info("Ditampilkan maks. 500 baris agar performa tetap
optimal.")

with tab2:
    st.subheader("Data Agregat per Produk (Alternatif Restocking)")
    st.markdown(
        "Setiap baris = 1 alternatif produk. Nilai merupakan
**agregat** dari seluruh transaksi produk tersebut. "
        "Produk dengan skor AHP tertinggi akan direkomendasikan untuk
di-restock terlebih dahulu."
    )
    st.dataframe(df_prod.style.format({
        "total_penjualan": "{:,.0f}",
        "rata_profit": "{:,.0f}",
        "rata_margin": "{:.2f}",
        "total_qty": "{:,.0f}",
        "rata_diskon": "{:.2f}",
    })), use_container_width=True, height=450)

# =====
# PAGE 2 - HITUNG SPK
# =====
elif page == "Hitung SPK":
    st.title("Perhitungan SPK - Metode AHP")
    st.markdown("""
    **Tujuan:** Menentukan prioritas produk yang perlu di-restock
    terlebih dahulu
    berdasarkan performa penjualan, profitabilitas, dan efisiensi
    diskon.

    **Langkah AHP:**
    1. Tentukan bobot kriteria lewat **matriks perbandingan berpasangan**
    (Pairwise Comparison).
    2. Hitung vektor prioritas (bobot) dan uji konsistensi (**CR <
    0.10**).
    3. Normalisasi nilai alternatif, lalu hitung **skor akhir** =  $\sum$ (bobot
     $\times$  nilai ternormalisasi).
    """)

    st.markdown("---")

```

```

# — Input Jenis Kriteria —————
st.subheader("① Tentukan Jenis Kriteria")
st.caption("Benefit = semakin besar semakin baik | Cost = semakin kecil semakin baik")

col_types = st.columns(N)
crit_types = {}
default_types = ["benefit", "benefit", "benefit", "benefit", "cost"]
for i, (lbl, col_key) in enumerate(CRITERIA.items()):
    with col_types[i]:
        crit_types[col_key] = st.selectbox(
            lbl.split("-")[0].strip(),
            ["benefit", "cost"],
            index=0 if default_types[i] == "benefit" else 1,
            key=f"type_{col_key}",
        )

st.markdown("----")

# — Matriks Perbandingan Berpasangan —————
st.subheader("② Matriks Perbandingan Berpasangan (Pairwise Comparison)")
st.caption(
    "Isi sel **baris vs kolom** menggunakan skala Saaty (1-9). "
    "Diagonal otomatis = 1. Nilai reciprocal dihitung otomatis."
)

SCALE_OPTIONS = list(SAATY_SCALE.keys())
matrix_vals = np.ones((N, N))

rows = st.columns([1] * N)
# Header labels
header_cols = st.columns([1.5] + [1] * N)
header_cols[0].markdown("***Kriteria***")
for j in range(N):
    header_cols[j + 1].markdown(f"C{j+1}")

for i in range(N):
    row_cols = st.columns([1.5] + [1] * N)
    row_cols[0].markdown(f"C{i+1} {CRITERIA_LABELS[i].split('-')[1].strip()}")
    for j in range(N):
        if i == j:
            row_cols[j + 1].markdown("1")
            matrix_vals[i][j] = 1
        elif i < j:
            sel = row_cols[j + 1].selectbox(
                "", SCALE_OPTIONS,
                index=0,
                key=f"pair_{i}_{j}",
                label_visibility="collapsed",
            )
            matrix_vals[i][j] = SAATY_SCALE[sel]
            matrix_vals[j][i] = 1 / SAATY_SCALE[sel]
        else:
            val = matrix_vals[i][j]
            if val < 1:
                disp = f"1/{int(round(1/val))}"

```

```

        else:
            disp = str(int(val))
            row_cols[j + 1].markdown(f"*{disp}*")

st.markdown("---")

# — Top-N Filter —————
st.subheader("③ Filter Alternatif")
col_a, col_b = st.columns(2)
with col_a:
    top_n = st.slider("Tampilkan Top-N Produk", min_value=5,
max_value=26, value=10, step=1)
    with col_b:
        kat_filter = st.multiselect(
            "Filter Kategori Produk",
            options=sorted(df_prod["kategori"].unique().tolist()),
            default=sorted(df_prod["kategori"].unique().tolist()),
        )

st.markdown("---")

# — TOMBOL HITUNG —————
st.subheader("④ Jalankan Perhitungan")
hitung_btn = st.button("Hitung AHP Sekarang!", type="primary",
use_container_width=True)

if hitung_btn:
    st.session_state["matrix_vals"] = matrix_vals.copy()
    st.session_state["crit_types"] = crit_types.copy()
    st.session_state["top_n"] = top_n
    st.session_state["kat_filter"] = kat_filter

    # — Hitung bobot AHP
    weights, lambda_max, ci, cr = ahp_weights(matrix_vals)
    st.session_state["weights"] = weights
    st.session_state["lambda_max"] = lambda_max
    st.session_state["ci"] = ci
    st.session_state["cr"] = cr

    # — Filter alternatif
    df_alt = df_prod[df_prod["kategori"].isin(kat_filter)].copy()

    # — Normalisasi
    benefit_cols = [c for c, t in crit_types.items() if t ==
"benefit"]
    cost_cols = [c for c, t in crit_types.items() if t == "cost"]
    df_norm = normalize_minmax(df_alt[CRITERIA_COLS], benefit_cols,
cost_cols)

    # — Skor akhir
    skor = df_norm.values @ weights
    df_alt = df_alt.copy()
    df_alt["Skor AHP"] = skor
    df_alt["Peringkat"] = df_alt["Skor AHP"].rank(ascending=False,
method="min").astype(int)
    df_alt = df_alt.sort_values("Peringkat")

    st.session_state["df_alt"] = df_alt
    st.session_state["df_norm"] = df_norm

```

```

st.session_state["computed"] = True

st.success("Perhitungan selesai! Lihat hasil di halaman **Hasil
& Grafik**.")

# — Tampilkan ringkasan bobot & CR — selalu muncul jika sudah pernah
dihitung —
if st.session_state.get("computed"):
    saved_matrix = st.session_state["matrix_vals"]
    saved_weights = st.session_state["weights"]
    saved_lambda = st.session_state["lambda_max"]
    saved_ci = st.session_state["ci"]
    saved_cr = st.session_state["cr"]
    saved_crit_types = st.session_state["crit_types"]

    st.markdown("---")
    st.subheader("Hasil Matriks & Bobot Kriteria")

    # Matriks ternormalisasi
    col_sum = saved_matrix.sum(axis=0)
    norm_matrix = saved_matrix / col_sum
    df_matrix_raw = pd.DataFrame(
        saved_matrix,
        index=[f"C{i+1}" for i in range(N)],
        columns=[f"C{j+1}" for j in range(N)],
    )
    df_matrix_norm = pd.DataFrame(
        norm_matrix,
        index=[f"C{i+1}" for i in range(N)],
        columns=[f"C{j+1}" for j in range(N)],
    )

    col_m1, col_m2 = st.columns(2)
    with col_m1:
        st.markdown("***Matriks Perbandingan Berpasangan***")
        st.dataframe(df_matrix_raw.style.format("{:.4f}"),
use_container_width=True)
    with col_m2:
        st.markdown("***Matriks Ternormalisasi***")
        st.dataframe(df_matrix_norm.style.format("{:.4f}"),
use_container_width=True)

    df_weights = pd.DataFrame({
        "Kode": [f"C{i+1}" for i in range(N)],
        "Kriteria": CRITERIA_LABELS,
        "Tipe": [saved_crit_types[c] for c in CRITERIA_COLS],
        "Bobot (Prioritas)": saved_weights,
        "Bobot (%)": saved_weights * 100,
    })
    st.markdown("***Vektor Prioritas (Bobot Kriteria)***")
    st.dataframe(df_weights.style.format({
        "Bobot (Prioritas)": "{:.4f}",
        "Bobot (%)": "{:.2f}%",
    })), use_container_width=True)

    # Uji Konsistensi
    cr_status = "Konsisten (CR < 0.10)" if saved_cr < 0.10 else
"Tidak Konsisten (CR ≥ 0.10)"

```

```

st.markdown("***Uji Konsistensi (Consistency Ratio)***")
col_c1, col_c2, col_c3, col_c4 = st.columns(4)
col_c1.metric("λ max", f"{saved_lambda:.4f}")
col_c2.metric("CI", f"{saved_ci:.4f}")
col_c3.metric("RI (n=5)", f"{SAATY_RI[N]:.2f}")
col_c4.metric("CR", f"{saved_cr:.4f}", delta=cr_status,
delta_color="normal")

if saved_cr >= 0.10:
    st.warning("CR ≥ 0.10! Perbandingan berpasangan kurang
konsisten. Silakan revisi matriks.")

else:
    st.info("Isi matriks perbandingan, lalu klik tombol **Hitung AHP
Sekarang!**")

# =====
# PAGE 3 - HASIL & GRAFIK
# =====
elif page == "Hasil & Grafik":
    st.title("Hasil Perangkingan Prioritas Restocking")

    if not st.session_state.get("computed"):
        st.warning("Belum ada hasil. Silakan jalankan perhitungan di
halaman **Hitung SPK** terlebih dahulu.")
        st.stop()

    df_alt: pd.DataFrame = st.session_state["df_alt"]
    weights: np.ndarray = st.session_state["weights"]
    cr: float = st.session_state["cr"]
    top_n: int = st.session_state["top_n"]

    # — Tabel hasil —————
    st.subheader("Tabel Prioritas Restocking Produk")

    display_cols = ["Peringkat", "nama_produk", "kategori",
                    "total_penjualan", "rata_profit", "rata_margin",
                    "total_qty", "rata_diskon", "Skor AHP"]

    df_show = df_alt[display_cols].head(top_n).copy()
    df_show.columns = ["Peringkat", "Produk", "Kategori",
                      "Total Penjualan", "Avg Profit", "Avg Margin
(%)",
                      "Total Qty", "Avg Diskon (%)", "Skor AHP"]

    def color_rank(val):
        if val == 1: return "background-color:#FFD700; font-
weight:bold"
        if val == 2: return "background-color:#C0C0C0; font-
weight:bold"
        if val == 3: return "background-color:#CD7F32; font-
weight:bold"
        return ""

    st.dataframe(
        df_show.style
        .format({
            "Total Penjualan": "{:,.0f}",

```

```

        "Avg Profit":      "{:,.0f}",
        "Avg Margin (%)":  "{:,.2f}",
        "Total Qty":      "{:,.0f}",
        "Avg Diskon (%)":  "{:,.2f}",
        "Skor AHP":        "{:,.4f}",
    })
    .applymap(color_rank, subset=["Peringkat"]),
    use_container_width=True,
    height=420,
)

st.caption(
    f"CR = **{cr:.4f}** {'Konsisten' if cr < 0.10 else 'Tidak Konsisten'} | "
    f"Menampilkan Top-{top_n} dari {len(df_alt)} produk yang difilter."
)

st.markdown("---")

# — Grafik —————
st.subheader("Grafik Skor AHP per Produk")

df_chart = df_alt.head(top_n).sort_values("Skor AHP", ascending=True)

fig, ax = plt.subplots(figsize=(10, max(5, top_n * 0.5)))
fig.patch.set_facecolor("#0e1117")
ax.set_facecolor("#0e1117")

colors_map = {
    "Aksesoris": "#4C9BE8",
    "Footwear":  "#F4A261",
    "Apparel":   "#2A9D8F",
    "Home":      "#E76F51",
    "Stationery": "#E9C46A",
}
bar_colors = [colors_map.get(k, "#888") for k in df_chart["kategori"]]

bars = ax.barh(df_chart["nama_produk"], df_chart["Skor AHP"], color=bar_colors, edgecolor="none", height=0.6)

# Tambah nilai di ujung bar
for bar, val in zip(bars, df_chart["Skor AHP"]):
    ax.text(bar.get_width() + 0.002, bar.get_y() + bar.get_height() / 2,
            f"{val:.4f}", va="center", ha="left", fontsize=8, color="white")

ax.set_xlabel("Skor AHP", color="white", fontsize=10)
ax.set_title(f"Top-{top_n} Produk Prioritas Restocking berdasarkan AHP", color="white", fontsize=13, pad=12)
ax.tick_params(colors="white")
ax.spines[["top", "right", "left", "bottom"]].set_visible(False)
ax.xaxis.grid(True, color="#333", linewidth=0.5)
ax.set_axisbelow(True)

```

```

legend_handles = [mpatches.Patch(color=c, label=k) for k, c in
colors_map.items()]
ax.legend(handles=legend_handles, loc="lower right",
          facecolor="#1a1a2e",                      edgecolor="none",
labelcolor="white", fontsize=8)

plt.tight_layout()
st.pyplot(fig)

st.markdown("---")

# — Bobot kriteria pie —————
st.subheader("Distribusi Bobot Kriteria")
fig2, ax2 = plt.subplots(figsize=(7, 4))
fig2.patch.set_facecolor("#0e1117")
ax2.set_facecolor("#0e1117")

pie_colors = ["#4C9BE8", "#F4A261", "#2A9D8F", "#E76F51", "#E9C46A"]
wedges, texts, autotexts = ax2.pie(
    weights,
    labels=[f"C{i+1}" for i in range(N)],
    autopct="%1.1f%%",
    colors=pie_colors,
    startangle=140,
    textprops={"color": "white"},
    wedgeprops={"edgecolor": "#0e1117", "linewidth": 2},
)
for at in autotexts:
    at.set_fontsize(9)
ax2.set_title("Bobot Kriteria AHP", color="white", fontsize=12)
leg = ax2.legend(
    [f"C{i+1} - {CRITERIA_LABELS[i].split('-')[1].strip()}" for i in
range(N)],
    loc="best", fontsize=8, facecolor="#1a1a2e",
    edgecolor="none", labelcolor="white",
)
plt.tight_layout()
st.pyplot(fig2, use_container_width=False)

```

<

SPK Restocking Retail

Sistem Pendukung Keputusan Berbasis AHP Optimasi Keputusan Restocking Produk bagi Manajer Retail Indonesia

Navigne

Dataset

Hitung SPK

Hasil & Grafik

Jumlah Data

Total Transaksi

55,000

Jumlah Produk

26

Jumlah Kriteria

5

Perhitungan SPK – Metode AHP

Tujuan: Menentukan prioritas produk yang perlu di-restock terlebih dahulu berdasarkan performa penjualan, profitabilitas, dan efisiensi diskon.

Langkah AHP:

1. Tentukan bobot kriteria lewat matriks perbandingan berpasangan (Pairwise Comparison).

2. Hitung vektor prioritas (bobot) dan uji konsistensi (CR < 0.10).

3. Normalisasi nilai alternatif, lalu hitung skor akhir = $\sum(\text{bobot} \times \text{nilai ternormalisasi})$.

1 Tentukan Jenis Kriteria

Benefit = semakin besar semakin baik | Cost = semakin kecil semakin baik

C1

benefit

C2

benefit

C3

benefit

C4

benefit

C5

cost

2 Matriks Perbandingan Berpasangan (Pairwise Comparison)

Isi sel baris vs kolom menggunakan skala Saaty (1-5). Diagonal otomatis = 1. Nilai reciprocal dihitung otomatis.

Kriteria	C1	C2	C3	C4	C5
C1 Total Penjualan	1	1 - Sama penting	1 - Sama penting	1 - Sama penting	1 - Sama penting
C2 Rata-rata Profit	1	1	1 - Sama penting	1 - Sama penting	1 - Sama penting
C3 Rata-rata Margin (%)	1	1	1	1 - Sama penting	1 - Sama penting
C4 Total Qty Terjual	1	1	1	1	1 - Sama penting
C5 Rata-rata Diskon (%)	1	1	1	1	1

SPK Restocking Retail

Sistem Pendukung Keputusan Berbasis AHP Optimasi Keputusan Restocking Produk bagi Manajer Retail Indonesia

Navigne

Dataset

Hitung SPK

Hasil & Grafik

Jumlah Data

Total Transaksi

55,000

Jumlah Produk

26

Jumlah Kriteria

5

3 Filter Alternatif

Tampilkan Top N Produk

10

Filter Kategori Produk

Aksesoris

Apparec

Furniture

Home

Stationery

4 Jalankan Perhitungan

Hitung AHP Sekarang!

Perhitungan selesai! Lihat hasil di halaman Hasil & Grafik.

Hasil Matriks & Bobot Kriteria

Matriks Perbandingan Berpasangan

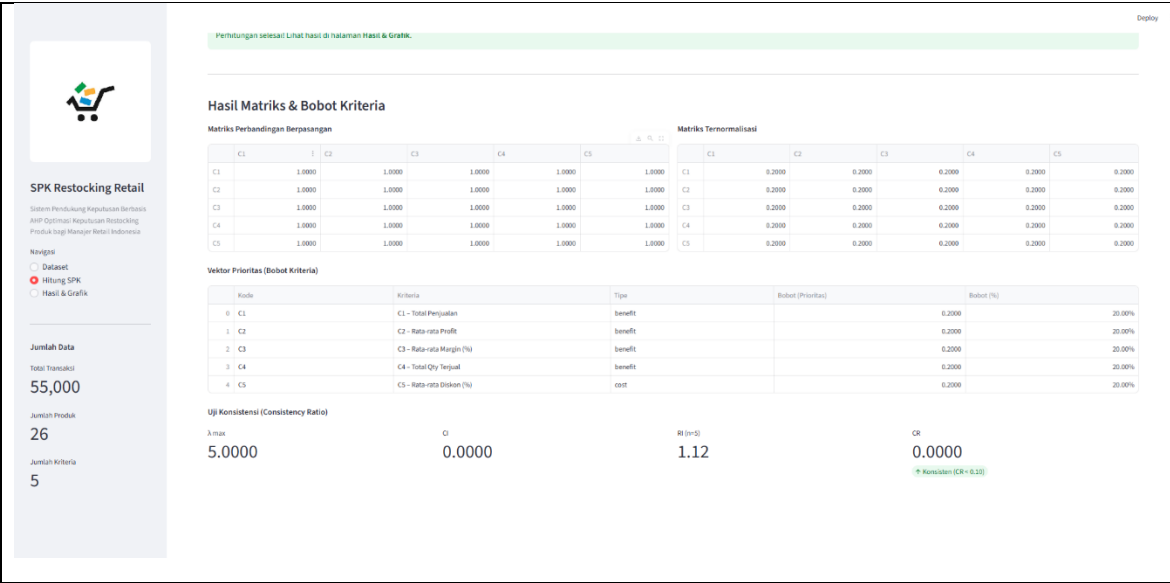
	C1	C2	C3	C4	C5
C1	1.0000	1.0000	1.0000	1.0000	1.0000
C2	1.0000	1.0000	1.0000	1.0000	1.0000
C3	1.0000	1.0000	1.0000	1.0000	1.0000
C4	1.0000	1.0000	1.0000	1.0000	1.0000
C5	1.0000	1.0000	1.0000	1.0000	1.0000

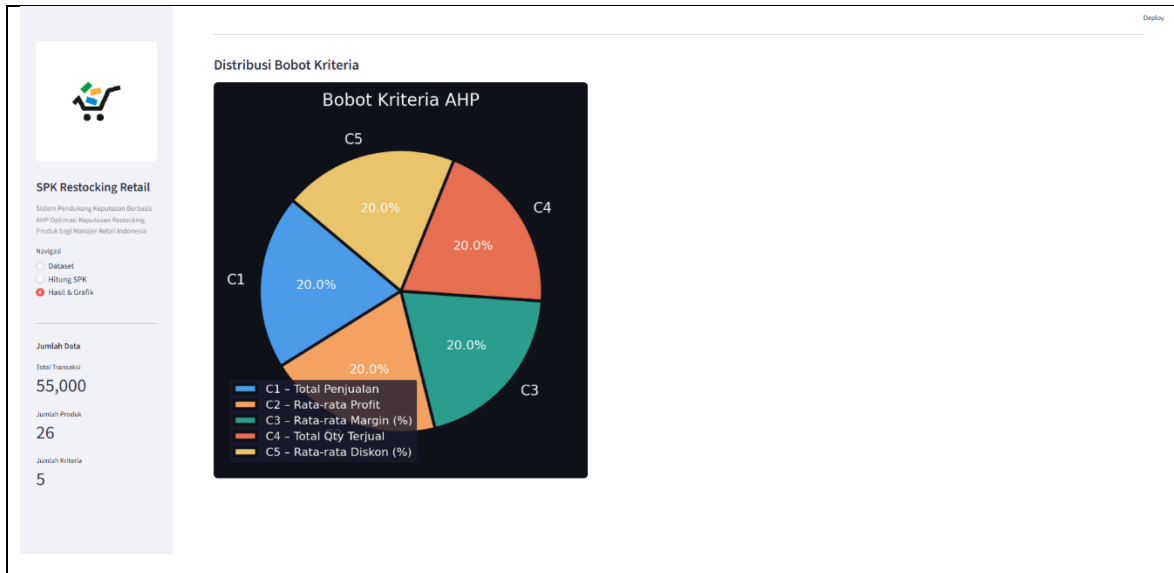
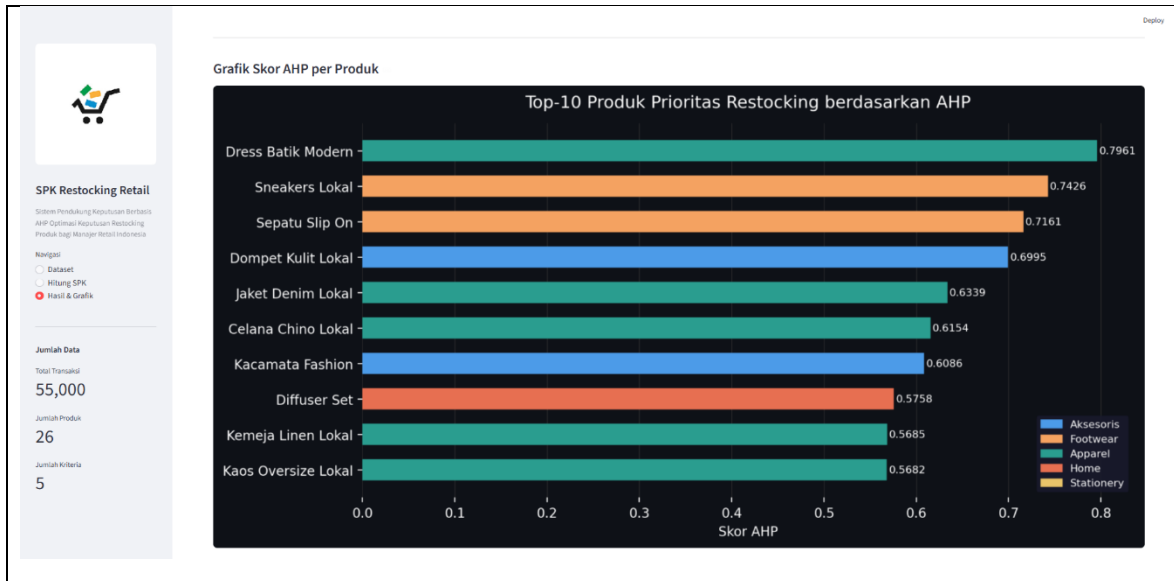
Matriks Ternormalisasi

	C1	C2	C3	C4	C5
C1	0.2000	0.2000	0.2000	0.2000	0.2000
C2	0.2000	0.2000	0.2000	0.2000	0.2000
C3	0.2000	0.2000	0.2000	0.2000	0.2000
C4	0.2000	0.2000	0.2000	0.2000	0.2000
C5	0.2000	0.2000	0.2000	0.2000	0.2000

Vektor Prioritas (Bobot Kriteria)

Kode	Kriteria	Tipe	Bobot (Prioritas)	Bobot (%)
0 C1	C1 - Total Penjualan	benefit		20.00%
1 C2	C2 - Rata-rata Profit	benefit		20.00%
2 C3	C3 - Rata-rata Margin (%)	benefit		20.00%





BAB III

JADWAL Pengerjaan dan Pembagian Tugas

3.1 Jadwal Pengerjaan

No	Kegiatan	2022				
		Minggu				
		1	2	3	4	dst
1	Identifikasi masalah dan studi literatur metode AHP					
2	Pengumpulan dan preprocessing dataset retail Indonesia					
3	Perancangan model SPK berbasis AHP					
4	Implementasi sistem menggunakan Streamlit					
5	Pengujian sistem dan evaluasi hasil					
6	Penyusunan laporan proyek akhir					

3.2 Pembagian Tugas

No	Nama	NIM	Deskripsi Tugas
1	Nayla Cahaya Zahira	123240024	- Perancangan model SPK berbasis AHP - Pengumpulan serta preprocessing dataset retail Indonesia - Pembuatan visualisasi data dan antarmuka sistem - Penyusunan laporan
2	Rifki Zakaria Yahya	123240200	- Studi literatur metode AHP dan penyusunan dasar teori - Implementasi algoritma AHP menggunakan Python dan Streamlit - Pengujian sistem serta analisis hasil perangkingan - Dokumentasi hasil implementasi sistem

BAB IV

KESIMPULAN DAN SARAN

4.2 Kesimpulan

Berdasarkan hasil pengerjaan proyek akhir ini, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Sistem Pendukung Keputusan (SPK) berbasis metode Analytical Hierarchy Process (AHP) berhasil dibangun dan diimplementasikan untuk membantu manajer retail dalam menentukan prioritas restocking produk secara objektif dan berbasis data.
2. Dataset retail Indonesia yang terdiri dari 55.000 transaksi dengan 26 produk dan 5 kategori berhasil diolah menjadi 26 alternatif keputusan menggunakan agregasi data per produk, dengan 5 kriteria penilaian yaitu Total Penjualan, Rata-rata Profit, Rata-rata Margin, Total Kuantitas Terjual, dan Rata-rata Diskon.
3. Metode AHP terbukti mampu mengakomodasi preferensi pengambil keputusan melalui mekanisme perbandingan berpasangan, dan menghasilkan bobot kriteria yang konsisten ($CR < 0,10$) sehingga hasil perangkingan dapat dipertanggungjawabkan secara ilmiah.
4. Sistem berhasil diimplementasikan dalam bentuk antarmuka grafis interaktif menggunakan Streamlit, memenuhi seluruh ketentuan yang ditetapkan meliputi navigasi sidebar, tabel interaktif, input bobot dinamis, tombol eksekusi, tabel hasil perangkingan, tampilan matriks perbandingan berpasangan, dan visualisasi grafik.

4.3 Saran

Beberapa saran untuk pengembangan sistem ini ke depannya adalah sebagai berikut:

1. Integrasi data real-time: Sistem dapat dikembangkan untuk terhubung langsung dengan database sistem kasir atau ERP toko sehingga data yang diproses selalu terkini tanpa perlu input manual file CSV.
2. Kombinasi metode: Sistem dapat diperkaya dengan menggabungkan AHP dengan metode lain seperti TOPSIS atau SAW untuk memberikan hasil perangkingan yang lebih komprehensif dan dapat dibandingkan antar metode.
3. Penambahan kriteria: Kriteria penilaian dapat diperluas dengan mempertimbangkan faktor seperti tren musiman, stok saat ini, lead time supplier, atau data kompetitor untuk menghasilkan rekomendasi yang lebih akurat.
4. Fitur ekspor laporan: Penambahan fitur untuk mengeksport hasil perangkingan dalam format PDF atau Excel akan meningkatkan kegunaan sistem bagi manajer dalam membuat laporan atau presentasi keputusan kepada pimpinan.

DAFTAR PUSTAKA

- Kaggle. (2024). *Retail Indonesia Dataset 55K Transactions*.
Diakses pada 25 Mei 2026, dari <https://www.kaggle.com/>
- Saaty, T. L. (1980). *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*. New York: McGraw-Hill.
- Streamlit Inc. (2025). *Streamlit Documentation*.
Diakses pada 25 Mei 2026, dari <https://docs.streamlit.io/>
- Turban, E., Aronson, J. E., Liang, T. P., & Sharda, R. (2007).
Decision Support Systems and Intelligent Systems (8th ed.). Upper Saddle River, NJ: Pearson Education.